Matrix operations: solutions

28-E01 Two triangular substitutions

Derivation

Multiplier $m_{21}=3$ gives $L=[1\ 0;3\ 1]$, $U=[1\ 2;0\ -2]$, $P=I$. Forward $Ly=b$ gives $y_1=5$, $y_2=11-3\cdot 5=-4$. Backward $Ux=y$ gives $x_2=-4/-2=2$, $x_1=5-2\cdot 2=1$. Original rows give $1+4=5$ and $3+8=11$.

Matrix$[0\ 1;1\ 0]$ squares to I, so it is its own inverse. Its first zero pivot invalidates that unpermuted elimination order, not invertibility. Swapping its rows gives $PA=I$ and $L=U=I$.

Conclusion and checks

Solution $x=(1,2)^T$ follows invariants on solved rows. A zero diagonal alone is not a singularity certificate; LUP's test uses an all-zero active pivot column.

28-E02 Schur complement and inverse

Derivation

Here $B=1$, $C=1$, $D=2$, $E=5$, so $S=5-2\cdot 1\cdot 1=3$. Formula entries are $1+1\cdot 1\cdot (1/3)\cdot 2\cdot 1=5/3$, $-1\cdot 1\cdot (1/3)=-1/3$, $-(1/3)\cdot 2\cdot 1=-2/3$, and $1/3$. Thus $A^{-1}=[5/3\ -1/3;-2/3\ 1/3]$.

Product entries are $5/3-2/3=1$, $-1/3+1/3=0$, $10/3-10/3=0$, $-2/3+5/3=1$. Singular B has no B^{-1}, so the formula cannot be applied in hopes of cancellation. For invertible $A=[0\ 1;1\ 0]$, $B=0$ requires another permutation or partition.

Conclusion and checks

$S=3$, $\det A=3$, and the stated inverse checks. Scalar blocks commute here, but general block matrices retain factor order.

28-E03 A line fit with nonzero residual

Derivation

$A=[1\ 0;1\ 1;1\ 2]$, $b=(0,2,1)^T$, $A^TA=[3\ 3;3\ 5]$, $A^Tb=(3,4)^T$. Subtraction gives $2m=1$, then $3c+3/2=3$, so $c=m=1/2$. Predictions $p=(1/2,1,3/2)$, residual $r=(-1/2,1,-1/2)$.

$A^Tr=(-1/2+1-1/2,0\cdot(-1/2)+1\cdot 1+2\cdot(-1/2))=(0,0)$, and squared norm $1/4+1+1/4=3/2$. These are vertical y-errors at fixed t, not shortest scatterpoint-to-line distances.

Matrix operations: solutions (continued)

Changing b to(1,2,3) leaves A unchanged, with independent columns and rank 2. New unique coefficients are c=m=1, residual 0. Data collinearity determines exact fit to this model, not rank of the design formed from t and basis functions.

Conclusion and checks

Minimum error changes from 3/2 to 0 while Gram stays identical with determinant 6>0. Nonzero residual and coefficient uniqueness have different conditions.

28-E04 Later permutations must swap earlier L entries

Derivation

First-column largest magnitude 2 swaps rows 1,3, so p=(3,2,1). Multipliers 1/2 and 0 leave active block[−1/2 −2;2 1]. Second-column largest magnitude 2 swaps current rows 2,3, so p=(3,1,2). Saved first-column multipliers become 0 in row 2 and 1/2 in row 3. New m_{32}=−1/4 and last pivot−7/4.

Final L=[1 0 0;0 1 0;1/2 −1/4 1], U=[2 3 4;0 2 1;0 0 −7/4]. Product rows[2,3,4],[0,2,1], [1,1,0] equal PA in original order 3,1,2.

Pb=(7,0,0), forward y=(7,0,−7/2), backward x=(1,−1,2). Original A gives(0,0,7). Incorrectly leaving L_{21}=1/2 makes product row 2 equal(1/2)(2,3,4)+(0,2,1)=(1,7/2,3), instead of PA row 2=(0,2,1).

Conclusion and checks

Both swaps include prior multipliers and synchronize p, Pb. Product of U diagonals is 2・2・(−7/4)=−7. Two swap signs cancel, giving original det A=−7.

28-E05 Both complexity reductions

Derivation

Multiplying T by the proposed inverse gives identity diagonal blocks. Block(1,2) is−B+B=0,(2,3) is−C+C=0, and(1,3) is BC+B(−C)+0=0. Lower blocks are 0. The upper-right inverse block is BC, without ever commuting to CB.

Construction writes $\Theta(n^2)$ entries, absorbed only with $I(n)=\Omega(n^2)$. The inverse call has order 3n, so I(3n)=O(I(n)) is needed to obtain O(I(n)). Arbitrary irregular cost functions do not permit omitting this assumption.

Matrix operations: solutions (continued)

In reverse, levelℓ has 2^ℓ subproblems, each with M(n/2^ℓ)$\leq$c^ℓM(n). Its work is O((2c)^ℓM(n));2c<1 makes the sum O(M(n)), including leaves. For SPD, leading and Schur blocks permit both inversions. For general real invertible A, Gram G=A^TA is SPD, and $G^{-1}A^TA=I$ proves $A^{-1}=G^{-1}A^T$. Extra products cost O(M(n)).

Conclusion and checks

The reductions specify dimension and exact-arithmetic regularity. Forming Gram squares conditioning, so the theoretical reduction is not a default floating-point solver.

28-E06 SPD and Cholesky

Derivation

Compute $C_{11}=\sqrt{4}=2$, $C_{21}=C_{31}=2/2=1$, $C_{22}=\sqrt{(5-1)}=2$, $C_{32}=(1-1\cdot 1)/2=0$, $C_{33}=\sqrt{(3-1^2-0^2)}=\sqrt{2}$. Thus C=[2 0 0;1 2 0;1 0 $\sqrt{2}$]. Product CC^T has rows[4,2,2],[2,5,1], [2,1,3], equal to A. Since C is invertible, every $z\neq 0$ has $z^TAz=||C^Tz||^2>0$.

After pivot 4, the Schur complement is

$$\begin{bmatrix}5 & 1\\ 1 & 3\end{bmatrix}-\begin{bmatrix}2\\ 2\end{bmatrix}\frac{1}{4}\begin{bmatrix}2 & 2\end{bmatrix}=\begin{bmatrix}4 & 0\\ 0 & 2\end{bmatrix}.$$

Its quadratic form $4u^2+2v^2$ is positive for nonzero(u, v), giving later positive pivots 4,2. By contrast, B=[1 2;2 1] has positive diagonals but z=(1,−1) gives $z^TBz=1-2-2+1=-2<0$. Its second elimination pivot is $1-2\cdot 2=-3$.

Conclusion and checks

SPD quantifies every nonzero vector. Original diagonals alone do not suffice. Here SPD Schur complements make every Cholesky step legal without row pivoting.

28-E07 Rank deficiency and error spaces

Derivation

Every prediction is s(1,1,1), where $s=c_1+c_2$. Squared error is $s^2+(2-s)^2+(1-s)^2=3(s-1)^2+2$, minimized at s=1. All coefficients are(t,1−t), unique prediction(1,1,1), residual(−1,1,0), squared norm 2, and $A^Tr=(0,0)$.

A nullspace coefficient change preserves s and error. Changing the sum to $1+\delta$ gives $2+3\delta^2$, strictly larger for $\delta\neq 0$. Rank is 1, and normal equations give only $c_1+c_2=1$; Gram is not invertible.

Matrix operations: solutions (continued)

Matrix diag(1,10^{-3}) has singular values 1,10^{-3}, so $\kappa_2=10^3$. Gram diag(1,10^{-6}) has condition 10^6. This measures possible directional perturbation amplification, not 10^6 executed operations. Factorization still has a cubic arithmetic bound in dimension n.

Conclusion and checks

Coefficients may be nonunique while prediction and orthogonal residual are unique. Sensitivity, uniqueness, and operation complexity require separate criteria.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrix operations: solutions (continued)

Verification code appendix: least_squares_check.py

```
"""Check the exact least-squares example and its orthogonality condition."""


def mat_vec(a, v):
    return [sum(row[j] * v[j] for j in range(len(v))) for row in a]


def dot(u, v):
    return sum(x * y for x, y in zip(u, v))


def solve_2x2(a, b):
    det = a[0][0] * a[1][1] - a[0][1] * a[1][0]
    assert det != 0
    return [
        (b[0] * a[1][1] - a[0][1] * b[1]) / det,
        (a[0][0] * b[1] - b[0] * a[1][0]) / det,
    ]


def run():
    a = [[1, 0], [1, 1], [1, 2]]
    b = [0, 2, 1]
    ata = [[dot([row[i] for row in a], [row[j] for row in a]) for j in range(2)] for i in
range(2)]
    atb = [dot([row[i] for row in a], b) for i in range(2)]
    assert ata == [[3, 3], [3, 5]]
    assert atb == [3, 4]
    solution = solve_2x2(ata, atb)
    assert solution == [0.5, 0.5]
    prediction = mat_vec(a, solution)
    residual = [target - fitted for target, fitted in zip(b, prediction)]
    assert prediction == [0.5, 1.0, 1.5]
    assert residual == [-0.5, 1.0, -0.5]
    assert [dot([row[i] for row in a], residual) for i in range(2)] == [0, 0]
    assert round(dot(residual, residual), 12) == 1.5
    print("PASS least-squares intercept=0.5 slope=0.5 residual=(-0.5,1,-0.5) AtR=0")
    return solution, residual


if __name__ == "__main__":
    run()
```

Matrix operations: solutions (continued)

Verification code appendix: matrix_operations_checks.py

```
"""Exact rational matrix identities, independently checked small determinants."""
from fractions import Fraction as F
from itertools import product,permutations
from math import prod
import random

SAMPLE=[[0,2,1],[1,1,0],[2,3,4]]


def transpose(a):return [list(row) for row in zip(*a)]
def identity(n):return [[F(i==j) for j in range(n)] for i in range(n)]
def multiply(a,b):return [[sum((x*y for x,y in zip(row,col)),F(0)) for col in zip(*b)] for
row in a]
def matvec(a,x):return [sum((v*w for v,w in zip(row,x)),F(0)) for row in a]
def subtract(a,b):return [[x-y for x,y in zip(r,s)] for r,s in zip(a,b)]


def determinant(a):
    n=len(a);answer=F(0)
    for p in permutations(range(n)):
        inversions=sum(p[i]>p[j] for i in range(n) for j in range(i+1,n))
        answer+=(-1)**inversions*prod((a[i][p[i]] for i in range(n)),start=F(1))
    return answer


def lup(a):
    n=len(a);work=[[F(x) for x in row] for row in a];order=list(range(n));trace=[];swaps=0
    for k in range(n):
        pivot=max(range(k,n),key=lambda i:abs(work[i][k]))
        if work[pivot][k]==0:raise ValueError('singular matrix')
        if pivot!=k:
            work[k],work[pivot]=work[pivot],work[k]
            order[k],order[pivot]=order[pivot],order[k];swaps+=1
        for i in range(k+1,n):
            work[i][k]/=work[k][k]
            for j in range(k+1,n):work[i][j]-=work[i][k]*work[k][j]
        trace.append(([row[:] for row in work],order[:]))
    lower=[[F(i==j) if i<=j else work[i][j] for j in range(n)] for i in range(n)]
```

Matrix operations: solutions (continued)

```
    upper=[[work[i][j] if i<=j else F(0) for j in range(n)] for i in range(n)]
    return lower,upper,order,trace,swaps


def solve(factor,b):
    lower,upper,order,_,_=factor;n=len(b);y=[F(0)]*n;x=y[:]
    for i in range(n):y[i]=F(b[order[i]])-sum((lower[i][j]*y[j] for j in range(i)),F(0))
    for i in range(n-1,-1,-1):x[i]=(y[i]-sum((upper[i][j]*x[j] for j in
range(i+1,n)),F(0)))/upper[i][i]
    return x,y


def inverse(a):
    n=len(a);factor=lup(a);columns=[solve(factor,[int(i==j) for i in range(n)])[0] for j in
range(n)]
    return transpose(columns)


def ldlt(a):
    n=len(a);lower=identity(n);diagonal=[F(0)]*n
    assert a==transpose(a)
    for j in range(n):
        diagonal[j]=a[j][j]-sum((lower[j][k]**2*diagonal[k] for k in range(j)),F(0))
        if diagonal[j]<=0:raise ValueError('not positive definite')
        for i in range(j+1,n):
            lower[i][j]=(a[i][j]-sum((lower[i][k]*lower[j][k]*diagonal[k] for k in
range(j)),F(0)))/diagonal[j]
    return lower,[[diagonal[i] if i==j else F(0) for j in range(n)] for i in range(n)]


def schur(a,cut):
    b=[row[:cut] for row in a[:cut]];c=[row[cut:] for row in a[:cut]]
    d=[row[:cut] for row in a[cut:]];e=[row[cut:] for row in a[cut:]]
    return subtract(e,multiply(multiply(d,inverse(b)),c))


def least_squares(a,b):
    at=transpose(a);gram=multiply(at,a);rhs=matvec(at,b)
    x,y=solve(lup(gram),rhs);prediction=matvec(a,x);residual=[F(u)-v for u,v in
zip(b,prediction)]
```

Matrix operations: solutions (continued)

```
    return x,prediction,residual,gram,rhs


def main():
    count=0;regular=0;inversions=0
    for n in range(1,4):
        for values in product([-1,0,1],repeat=n*n):
            a=[list(values[i*n:(i+1)*n]) for i in range(n)];det=determinant(a)
            try:factor=lup(a)
            except ValueError:
                assert det==0;count+=1;continue
            assert det!=0;regular+=1
            l,u,p,trace,swaps=factor
            assert multiply(l,u)==[a[i] for i in p]
            assert (-1)**swaps*prod((u[i][i] for i in range(n)),start=F(1))==det
            expected=[F(i+1) for i in range(n)];b=matvec(a,expected)
            assert solve(factor,b)[0]==expected
            if count%17==0 or n<3:
                inv=inverse(a);assert
multiply(a,inv)==identity(n)==multiply(inv,a);inversions+=1
            count+=1
    factor=lup(SAMPLE);l,u,p,trace,swaps=factor
    assert p==[2,0,1] and swaps==2
    assert l==[[1,0,0],[0,1,0],[F(1,2),F(-1,4),1]]
    assert u==[[2,3,4],[0,2,1],[0,0,F(-7,4)]]
    assert solve(factor,[0,0,7])==([1,-1,2],[7,0,F(-7,2)])
    rng=random.Random(28);spd_cases=0;least_cases=0
    for n in range(1,7):
        for _ in range(40):
            c=[[F(rng.randrange(-3,4)) for _ in range(n)] for _ in range(n)]
            a=multiply(transpose(c),c)
            for i in range(n):a[i][i]+=1
            lower,diag=ldlt(a);assert multiply(multiply(lower,diag),transpose(lower))==a
            if n>1:
                for cut in range(1,n):
                    s=schur(a,cut);ls,ds=ldlt(s);assert
multiply(multiply(ls,ds),transpose(ls))==s
            spd_cases+=1
    for m in range(2,8):
```

Matrix operations: solutions (continued)

```
        for n in range(1,min(4,m+1)):
            for _ in range(12):
                a=[[F(rng.randrange(-3,4)) for _ in range(n)] for _ in range(m)]
                b=[F(rng.randrange(-3,4)) for _ in range(m)]
                try:x,prediction,residual,gram,rhs=least_squares(a,b)
                except ValueError:continue
                assert matvec(transpose(a),residual)==[0]*n
                base=sum(v*v for v in residual)
                for direction in product([-1,0,1],repeat=n):
                    changed=[v+w for v,w in zip(x,direction)]
                    updated=[y-z for y,z in zip(b,matvec(a,changed))]
                    assert sum(v*v for v in updated)==base+sum(v*v for v in
matvec(a,direction))
                least_cases+=1
    a=[[1,0],[1,1],[1,2]];b=[0,2,1]
    assert least_squares(a,b)==([F(1,2),F(1,2)],[F(1,2),1,F(3,2)],
                                [F(-1,2),1,F(-1,2)],[[3,3],[3,5]],[3,4])
    plus=multiply(inverse(multiply(transpose(a),a)),transpose(a))
    assert plus==[[F(5,6),F(1,3),F(-1,6)],[F(-1,2),0,F(1,2)]]
    projection=multiply(a,plus)
    assert multiply(plus,a)==identity(2)
    assert multiply(projection,projection)==projection==transpose(projection)
    assert matvec(projection,b)==[F(1,2),1,F(3,2)]
    chol_a=[[4,2,2],[2,5,1],[2,1,3]]
    assert ldlt(chol_a)==([[1,0,0],[F(1,2),1,0],[F(1,2),0,1]],[[4,0,0],[0,4,0],[0,0,2]])
    assert inverse([[1,1],[2,5]])==[[F(5,3),F(-1,3)],[F(-2,3),F(1,3)]]
    # Without swapping the already-built L prefix, the sample factorization fails.
    wrong_l=[row[:] for row in l];wrong_l[1][0],wrong_l[2][0]=wrong_l[2][0],wrong_l[1][0]
    assert multiply(wrong_l,u)!=[SAMPLE[i] for i in p]
    print(f'PASS exact_small_matrices={count} nonsingular={regular}
sampled_two_sided_inverses={inversions} '
          f'SPD_and_all_leading_Schur_cases={spd_cases}
least_squares_orthogonality_and_Pythagoras={least_cases} '
          'L_prefix_swap_counterexample=True sample_x=1,-1,2 residual_squared=3/2')


if __name__=='__main__':main()
```

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study